REVISION CHEAT SHEET:

SQL SERVER AGGREGATE FUNCTIONS, GROUP BY, HAVING, AND ADVANCED GROUPING

Interview-ready notes • examples • SQL • diagrams

Aggregate functions are used when we want to answer questions about **many rows together**.

For example:

EXAMPLE / FLOW

```
How many orders do we have?

What is total revenue?

What is average salary?

What is the highest price?

What is the lowest salary?

How many customers are in each state?

What is total sales for each month?
```

Instead of returning every individual row, aggregation lets SQL Server calculate a **summary**.

Basic Idea of Aggregation

Suppose we have:

OrderIdDepartmentAmount		
1	IT	100
2	IT	200
3	HR	150
4	HR	250

Normal query:

SQL

```
SELECT Amount
FROM Orders;
```

returns:

EXAMPLE / FLOW

```
100
200
150
250
```

Aggregate query:

```sql
SELECT SUM(Amount)
FROM Orders;
```

returns:

```
700
```

Instead of returning four individual amounts, SQL Server summarized them into one result.

Aggregate Function vs Scalar Function

This difference is very important.

A **scalar function** works on one value and normally returns one result for each input row.

Example:

```sql
SELECT UPPER(EmployeeName)
FROM Employees;
```

If there are 100 employees:

```
100 input rows
→
100 results
```

An **aggregate function** works across multiple rows and produces a summary.

Example:

```sql
SELECT AVG(Salary)
FROM Employees;
```

If there are 100 employees:

```
100 input rows
→
1 summarized value
```

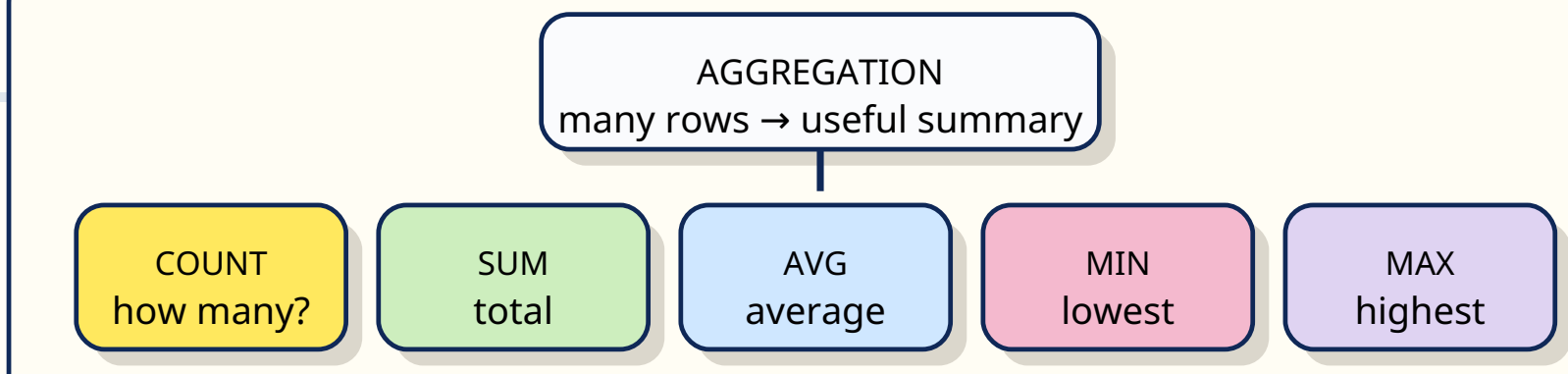
Easy memory:

EXAMPLE / FLOW

```
Scalar Function
=
One row/value → one result

Aggregate Function
=
Many rows → summarized result
```

Σ Core Aggregate Functions



The most important aggregate functions are:

EXAMPLE / FLOW

```
COUNT
SUM
AVG
MIN
MAX
```

SQL Server also provides functions such as:

EXAMPLE / FLOW

```
COUNT_BIG
STRING_AGG
CHECKSUM_AGG
STDEV
STDEVP
VAR
VARP
```

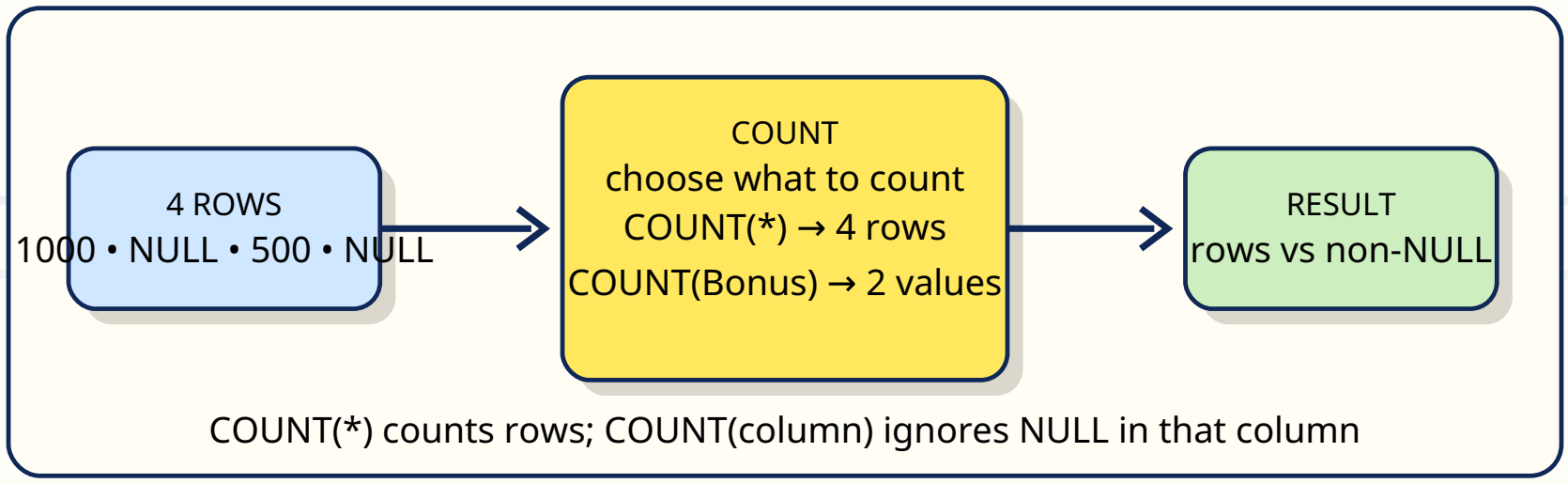
Later we will also use:

EXAMPLE / FLOW

```
GROUPING
GROUPING_ID
```

with advanced grouping.

COUNT(*)



Simple Definition

`COUNT(*)` counts **rows**.

Example:

```
SQL

SELECT COUNT(*)
FROM Employees;
```

If the table has:

```
EXAMPLE / FLOW

100 rows
```

the result is:

```
EXAMPLE / FLOW

100
```

Important COUNT(*) Rule

`COUNT(*)` counts the row even when some columns contain `NULL`.

Suppose:

EmployeeIdBonus	
1	1000
2	NULL
3	500
4	NULL

Then:

SQL

```
SELECT COUNT(*)
FROM Employees;
```

returns:

EXAMPLE / FLOW

```
4
```

because there are four rows.

When to Use COUNT(*)

Use it when you want:

EXAMPLE / FLOW

```
Total rows

Total employees

Total orders

Total customers

Rows per department
```

Example:

SQL

```
SELECT COUNT(*) AS TotalEmployees
FROM Employees;
```

COUNT(column)

Simple Definition

`COUNT(column)` counts only rows where that column is **not NULL**.

Using the same data:

EmployeeIdBonus	
1	1000
2	NULL
3	500
4	NULL

SQL

```
SELECT COUNT(Bonus)
FROM Employees;
```

returns:

EXAMPLE / FLOW

```
2
```

because only two employees have a bonus value.

COUNT(*) vs COUNT(column)

This is one of the most important interview concepts.

EXAMPLE / FLOW

```
COUNT(*)
→ Counts rows

COUNT(column)
→ Counts non-NULL values
```

Example:

SQL

```sql
SELECT
    COUNT(*) AS TotalEmployees,
    COUNT(Bonus) AS EmployeesWithBonus
FROM Employees;
```

Possible result:

TotalEmployeesEmployeesWithBonus	
4	2

COUNT(DISTINCT column)

Simple Definition

Counts unique non-NULL values.

Suppose:

EXAMPLE / FLOW

```
Department
----------
IT
IT
HR
HR
Finance
NULL
```

Query:

SQL

```sql
SELECT COUNT(DISTINCT Department)
FROM Employees;
```

Result:

EXAMPLE / FLOW

```
3
```

because the distinct non-NULL departments are:

EXAMPLE / FLOW

```
IT
HR
Finance
```

COUNT DISTINCT Example

How many unique customers placed an order?

SQL

```sql
SELECT COUNT(DISTINCT CustomerId)
FROM Orders;
```

If customer 10 placed twenty orders, customer 10 is still counted only once.

COUNT_BIG

`COUNT_BIG` works like `COUNT`, but returns:

EXAMPLE / FLOW

```
BIGINT
```

instead of:

EXAMPLE / FLOW

```
INT
```

Example:

SQL

```sql
SELECT COUNT_BIG(*)
FROM VeryLargeTable;
```

Use it when a result could exceed the `INT` range.

For ordinary application tables:

SQL

```sql
COUNT(*)
```

is normally enough.

SUM

Simple Definition

SUM adds numeric values together.

Suppose:

EXAMPLE / FLOW

```
Amount
------
100
200
300
```

Query:

SQL

```
SELECT SUM(Amount)
FROM Orders;
```

Result:

EXAMPLE / FLOW

```
600
```

SUM Example

SQL

```
SELECT
    SUM(TotalAmount) AS TotalRevenue
FROM Orders;
```

Possible result:

EXAMPLE / FLOW

```
TotalRevenue
------------
1250000.00
```

When to Use SUM

Use for:

EXAMPLE / FLOW

```
Revenue
Salary
Quantity
Tax
Payments
Discount amounts
Units sold
Account values
```

SUM and NULL

Most SQL aggregate functions ignore `NULL`.

Suppose:

EXAMPLE / FLOW

```
Bonus
-----
1000
NULL
500
NULL
```

SQL

```
SELECT SUM(Bonus)
FROM Employees;
```

returns:

EXAMPLE / FLOW

```
1500
```

The NULL values are ignored.

SUM of a Calculated Expression

Aggregate functions can calculate expressions.

Suppose:

EXAMPLE / FLOW

```
Quantity
UnitPrice
```

exist in `OrderItems`.

Instead of storing total line amount:

SQL

```
SELECT
    SUM(Quantity * UnitPrice) AS Revenue
FROM OrderItems;
```

SQL Server first calculates:

EXAMPLE / FLOW

```
Quantity × UnitPrice
```

for each row and then sums those results.

Example:

Quantity	UnitPrice
2	100
3	50

Calculation:

EXAMPLE / FLOW

```
2 × 100 = 200

3 × 50 = 150
```

Total:

EXAMPLE / FLOW

```
350
```

AVG

Simple Definition

AVG calculates the arithmetic average.

Suppose:

EXAMPLE / FLOW

```
Salary
------
50000
60000
70000
```

Query:

SQL

```
SELECT AVG(Salary)
FROM Employees;
```

Conceptually:

EXAMPLE / FLOW

```
50000 + 60000 + 70000
---------------------
          3

= 60000
```

AVG and NULL

AVG ignores NULL values.

Suppose:

EXAMPLE / FLOW

```
Salary
------
50000
60000
NULL
70000
```

The calculation is:

EXAMPLE / FLOW

```
50000 + 60000 + 70000
---------------------
          3
```

not:

EXAMPLE / FLOW

```
divide by 4
```

Important AVG Data-Type Rule

Be careful when averaging integer columns.

For example:

SQL

```
SELECT AVG(Rating)
FROM Reviews;
```

if `Rating` is an `INT`, the resulting type may also behave as an integer result.

If you need decimal precision:

SQL

```
SELECT AVG(
    CAST(Rating AS DECIMAL(10,2))
)
FROM Reviews;
```

Example:

EXAMPLE / FLOW

```
Ratings
1
2
2
```

True mathematical average:

EXAMPLE / FLOW

```
1.666...
```

Using an appropriate decimal conversion preserves the fractional result.

MIN

Simple Definition

`MIN` returns the smallest value.

Example:

```sql
SELECT MIN(Salary)
FROM Employees;
```

Possible result:

EXAMPLE / FLOW

```
45000
```

MIN with Dates

`MIN` also works with dates.

```sql
SELECT MIN(OrderDate)
FROM Orders;
```

returns the earliest order date.

MIN with Strings

`MIN` can work with supported character values according to their sorting/collation rules.

Example:

```sql
SELECT MIN(EmployeeName)
FROM Employees;
```

may return the alphabetically smallest value.

MAX

Simple Definition

`MAX` returns the largest value.

Example:

```sql
SELECT MAX(Salary)
FROM Employees;
```

Possible result:

EXAMPLE / FLOW

```
200000
```

MAX with Dates

SQL

```
SELECT MAX(OrderDate)
FROM Orders;
```

returns the latest order date.

Core Aggregate Cheat Sheet

FunctionPurposeNULL Behavior		
`COUNT(*)`	Count rows	Counts rows containing NULLs
`COUNT(column)`	Count populated values	Ignores NULL
`COUNT(DISTINCT column)`	Count unique values	Ignores NULL
`COUNT_BIG`	Large row count	Same logic as COUNT
`SUM`	Add values	Ignores NULL
`AVG`	Average	Ignores NULL
`MIN`	Lowest value	Ignores NULL
`MAX`	Highest value	Ignores NULL

Multiple Aggregate Functions in One Query

You can calculate many summaries at once.

SQL

```
SELECT
    COUNT(*) AS EmployeeCount,
    SUM(Salary) AS TotalSalary,
    AVG(Salary) AS AverageSalary,
    MIN(Salary) AS MinimumSalary,
    MAX(Salary) AS MaximumSalary
FROM Employees;
```

Possible result:

EmployeeCountTotalSalaryAverageSalaryMinimumSalaryMaximumSalary				
100	7000000	70000	40000	180000

One scan of the data can produce several aggregate results.

STRING_AGG

Simple Definition

`STRING_AGG` combines values from several rows into one string.

Suppose:

EXAMPLE / FLOW

```
Department
----------
IT
HR
Finance
```

Query:

SQL

```sql
SELECT STRING_AGG(
    DepartmentName,
    ', '
)
FROM Departments;
```

Result:

EXAMPLE / FLOW

```
IT, HR, Finance
```

STRING_AGG with GROUP BY

Suppose employees belong to departments.

SQL

```sql
SELECT
    DepartmentId,
    STRING_AGG(EmployeeName, ', ')
        AS Employees
FROM Employees
GROUP BY DepartmentId;
```

Possible result:

DepartmentIdEmployees	
1	John, David, Sarah
2	Priya, Michael

STRING_AGG Ordering

You can request ordering within the aggregated string.

Example:

SQL

```
SELECT
    DepartmentId,
    STRING_AGG(EmployeeName, ', ')
        WITHIN GROUP
        (
            ORDER BY EmployeeName
        ) AS Employees
FROM Employees
GROUP BY DepartmentId;
```

STRING_AGG and NULL

`STRING_AGG` ignores NULL input values and normally doesn't add a separator for ignored NULL values.

If you need a replacement:

SQL

```
STRING_AGG(
    ISNULL(EmployeeName, 'Unknown'),
    ', '
)
```

CHECKSUM_AGG

Simple Definition

`CHECKSUM_AGG` calculates a checksum over a group of values.

Example:

SQL

```
SELECT CHECKSUM_AGG(
    CHECKSUM(
        EmployeeId,
        Salary
    )
)
FROM Employees;
```

It can help detect whether data in a group **may have changed**.

Important CHECKSUM_AGG Rule

Do not treat it as:

EXAMPLE / FLOW

```
Cryptographic hash
```

or:

EXAMPLE / FLOW

```
Guaranteed proof that data is identical
```

Different data can theoretically produce the same checksum.

It is useful for lightweight change-detection scenarios, not strong security or guaranteed equality.

STDEV

STDEV calculates **sample standard deviation**.

Example:

SQL

```
SELECT STDEV(Salary)
FROM Employees;
```

Standard deviation tells you approximately how spread out values are around their average.

Low standard deviation:

EXAMPLE / FLOW

```
Values are relatively close together.
```

High standard deviation:

EXAMPLE / FLOW

```
Values are more widely spread.
```

STDEVP

STDEVP calculates **population standard deviation**.

Difference:

EXAMPLE / FLOW

```
STDEV
→ Sample

STDEVP
→ Entire population
```

Use the correct one based on what your data represents.

VAR

VAR calculates **sample statistical variance**.

SQL

```
SELECT VAR(Salary)
FROM Employees;
```

Variance measures spread mathematically.

VARP

VARP calculates **population variance**.

EXAMPLE / FLOW

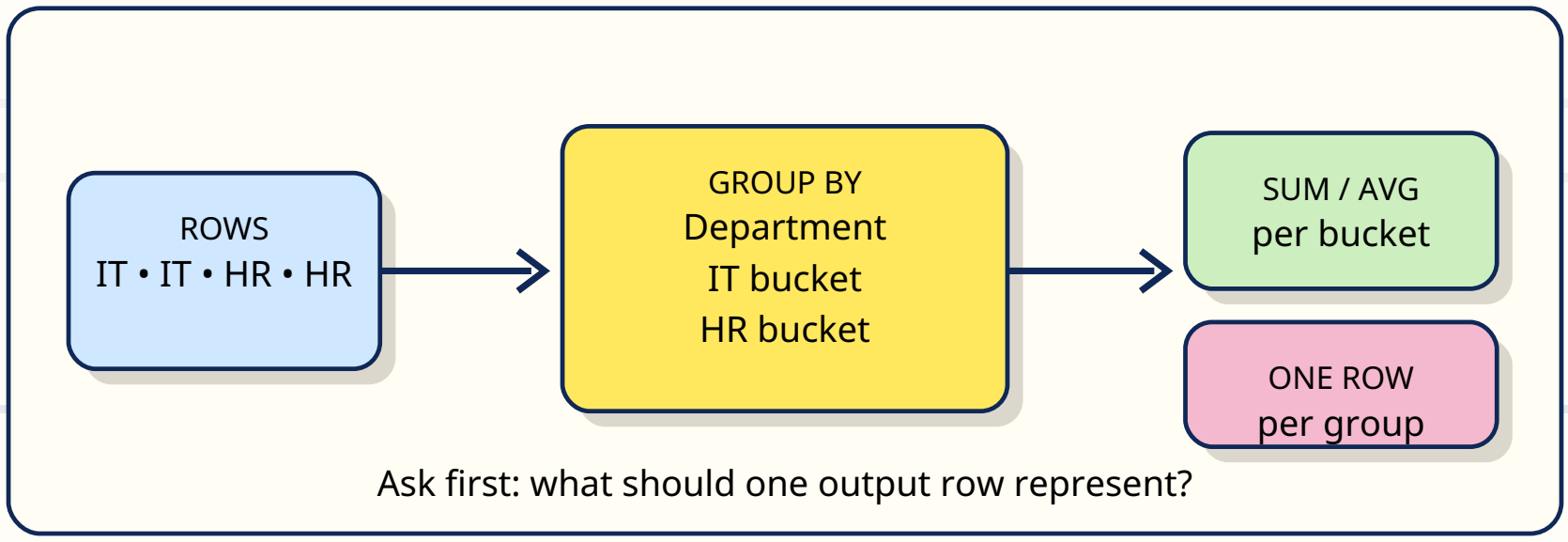
```
VAR
→ Sample variance

VARP
→ Population variance
```

Statistical Aggregate Cheat Sheet

FunctionPurpose	
STDEV	Sample standard deviation
STDEVP	Population standard deviation
VAR	Sample variance
VARP	Population variance
CHECKSUM_AGG	Aggregate checksum
STRING_AGG	Combine strings

GROUP BY



Aggregate functions become much more powerful with:

SQL

```
GROUP BY
```

Without `GROUP BY`:

SQL

```
SELECT SUM(Salary)
FROM Employees;
```

returns total salary for the entire table.

With:

```
GROUP BY Department
```

you can calculate salary separately for every department.

Simple GROUP BY

Suppose:

EmployeeDepartmentSalary		
John	IT	70000
David	IT	80000
Priya	HR	60000
Sarah	HR	65000

Query:

```
SELECT
    Department,
    SUM(Salary) AS TotalSalary
FROM Employees
GROUP BY Department;
```

Result:

DepartmentTotalSalary	
IT	150000
HR	125000

SQL Server logically creates:

```
IT group

HR group
```

and performs `SUM` separately for each group.

GROUP BY Simple Definition

`GROUP BY` puts rows having the same grouping value into groups so an aggregate calculation can be performed separately for each group.

GROUP BY One Column

Example:

SQL

```
SELECT
    Department,
    COUNT(*) AS EmployeeCount
FROM Employees
GROUP BY Department;
```

Possible result:

DepartmentEmployeeCount	
IT	20
HR	8
Finance	10

GROUP BY Multiple Columns

You can group using multiple columns.

Example:

SQL

```
SELECT
    Department,
    City,
    COUNT(*) AS EmployeeCount
FROM Employees
GROUP BY
    Department,
    City;
```

SQL Server creates a separate group for every unique combination:

EXAMPLE / FLOW

```
IT + New York

IT + Dallas

HR + New York

HR + Dallas
```

Multiple Column Example

Suppose:

DepartmentCityCount		
IT	New York	10
IT	Dallas	5
HR	New York	4
HR	Dallas	2

Each:

EXAMPLE / FLOW

```
Department + City
```

combination becomes one group.

Important GROUP BY Rule

When a query uses aggregation, every selected expression normally must either:

EXAMPLE / FLOW

```
Be aggregated
```

or:

EXAMPLE / FLOW

```
Appear appropriately in GROUP BY
```

Wrong:

SQL

```sql
SELECT
    Department,
    EmployeeName,
    AVG(Salary)
FROM Employees
GROUP BY Department;
```

Why wrong?

Because:

EXAMPLE / FLOW

```
Department
```

identifies the group.

But each department can have many:

EXAMPLE / FLOW

```
EmployeeName
```

values.

SQL Server doesn't know which employee name should represent the group.

Correct Version

```sql
SELECT
    Department,
    AVG(Salary) AS AverageSalary
FROM Employees
GROUP BY Department;
```

Or group by both:

```sql
SELECT
    Department,
    EmployeeName,
    AVG(Salary) AS AverageSalary
FROM Employees
GROUP BY
    Department,
    EmployeeName;
```

But that means something different.

Why Aggregate Columns Don't Need GROUP BY

This is valid:

```sql
SELECT
    Department,
    COUNT(*) AS EmployeeCount,
    SUM(Salary) AS TotalSalary
FROM Employees
GROUP BY Department;
```

Because:

EXAMPLE / FLOW

```
Department
→ Grouping column

COUNT(*)
→ Aggregate

SUM(Salary)
→ Aggregate
```

GROUP BY Expressions

You can group by an expression.

Example:

SQL

```
SELECT
    YEAR(OrderDate) AS OrderYear,
    COUNT(*) AS TotalOrders
FROM Orders
GROUP BY YEAR(OrderDate);
```

Result:

OrderYearTotalOrders	
2024	15000
2025	22000
2026	31000

Important Alias Rule

This generally doesn't work in the same query level:

SQL

```
SELECT
    YEAR(OrderDate) AS OrderYear,
    COUNT(*)
FROM Orders
GROUP BY OrderYear;
```

because the `SELECT` alias isn't generally available to `GROUP BY` at that same logical processing stage.

Use:

SQL

```
GROUP BY YEAR(OrderDate)
```

instead.

GROUP BY Dates

A very common requirement is:

EXAMPLE / FLOW

```
Sales by year

Sales by month

Orders by day
```

Group by Year

SQL

```
SELECT
    YEAR(OrderDate) AS OrderYear,
    COUNT(*) AS TotalOrders
FROM Orders
GROUP BY YEAR(OrderDate)
ORDER BY OrderYear;
```

Group by Month Number

SQL

```
SELECT
    MONTH(OrderDate) AS OrderMonth,
    COUNT(*) AS TotalOrders
FROM Orders
GROUP BY MONTH(OrderDate)
ORDER BY OrderMonth;
```

Be careful:

If your table contains several years, grouping only by month combines:

EXAMPLE / FLOW

```
January 2025
+
January 2026
```

into the same group.

Better Monthly Grouping Across Years

SQL

```
SELECT
    YEAR(OrderDate) AS OrderYear,
    MONTH(OrderDate) AS OrderMonth,
    COUNT(*) AS TotalOrders
FROM Orders
GROUP BY
    YEAR(OrderDate),
    MONTH(OrderDate)
ORDER BY
    OrderYear,
    OrderMonth;
```

Monthly Date Bucket

A useful design is to create the first day of the month:

SQL

```
SELECT
    DATEFROMPARTS(
        YEAR(OrderDate),
        MONTH(OrderDate),
        1
    ) AS OrderMonth,
    SUM(TotalAmount) AS Revenue
FROM Orders
GROUP BY
    DATEFROMPARTS(
        YEAR(OrderDate),
        MONTH(OrderDate),
        1
    )
ORDER BY OrderMonth;
```

Possible output:

EXAMPLE / FLOW

```
2026-01-01
2026-02-01
2026-03-01
```

Each value represents its month.

GROUP BY CASE

You can group rows into custom categories.

Example:

SQL

```
SELECT
    CASE
        WHEN Salary >= 100000
            THEN 'High'

        WHEN Salary >= 60000
            THEN 'Medium'

        ELSE 'Low'
    END AS SalaryCategory,

    COUNT(*) AS EmployeeCount

FROM Employees

GROUP BY
    CASE
        WHEN Salary >= 100000
            THEN 'High'

        WHEN Salary >= 60000
            THEN 'Medium'

        ELSE 'Low'
    END;
```

This groups employees into:

EXAMPLE / FLOW

```
High
Medium
Low
```

Conditional Aggregation

Sometimes you don't need several GROUP BY queries.

You can combine `CASE` with aggregate functions.

Example:

SQL

```
SELECT
    SUM(
        CASE
            WHEN Status = 'Paid'
            THEN 1
            ELSE 0
        END
    ) AS PaidOrders,

    SUM(
        CASE
            WHEN Status = 'Pending'
            THEN 1
            ELSE 0
        END
    ) AS PendingOrders

FROM Orders;
```

Possible result:

PaidOrdersPendingOrders	
800	120

Conditional SUM

SQL

```
SELECT
    SUM(
        CASE
            WHEN Status = 'Paid'
            THEN TotalAmount
            ELSE 0
        END
    ) AS PaidRevenue
FROM Orders;
```

This is an extremely common reporting pattern.

HAVING

Simple Definition

`HAVING` filters **groups after aggregation**.

Suppose you calculate:

EXAMPLE / FLOW

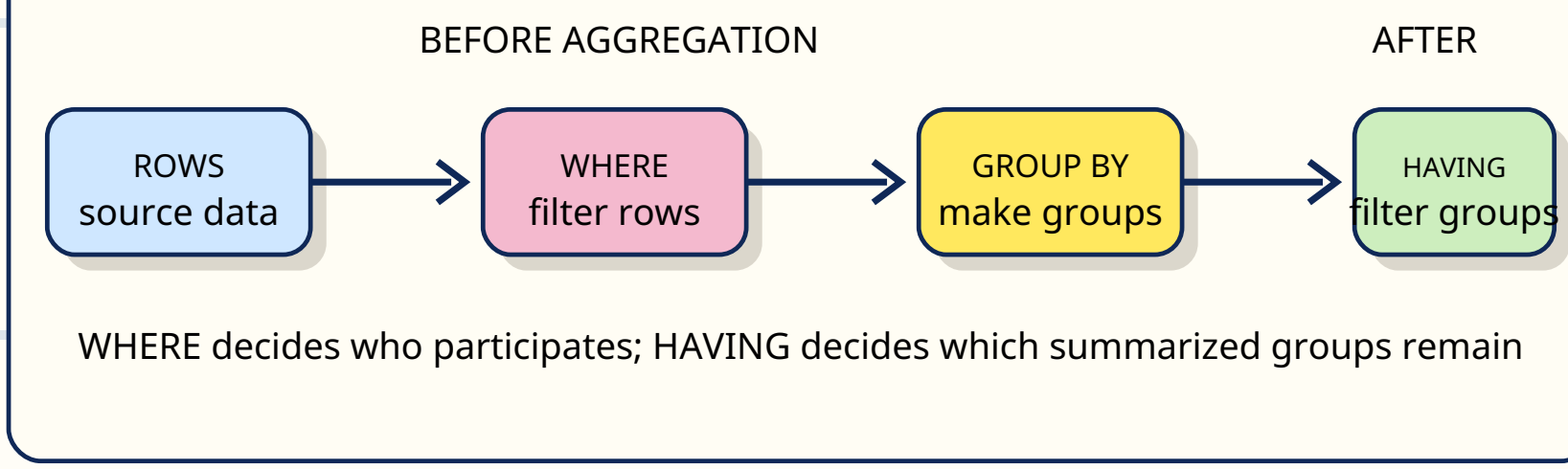
```
Employee count by department
```

but only want departments with more than 10 employees.

SQL

```
SELECT
    Department,
    COUNT(*) AS EmployeeCount
FROM Employees
GROUP BY Department
HAVING COUNT(*) > 10;
```

WHERE vs HAVING



This is one of the most important concepts in aggregation.

EXAMPLE / FLOW

```
WHERE
→ Filters rows BEFORE grouping

HAVING
→ Filters groups AFTER aggregation
```

WHERE Example

SQL

```
SELECT
    Department,
    AVG(Salary) AS AverageSalary
FROM Employees
WHERE IsActive = 1
GROUP BY Department;
```

Process conceptually:

EXAMPLE / FLOW

```
All employees
      ↓
WHERE IsActive = 1
      ↓
Only active employees
      ↓
GROUP BY Department
      ↓
AVG Salary
```

Inactive employees never enter the aggregation.

HAVING Example

SQL

```sql
SELECT
    Department,
    AVG(Salary) AS AverageSalary
FROM Employees
GROUP BY Department
HAVING AVG(Salary) > 70000;
```

SQL Server first calculates average salary per department.

Then it keeps only groups where:

EXAMPLE / FLOW

```
AverageSalary > 70000
```

WHERE and HAVING Together

SQL

```sql
SELECT
    Department,
    COUNT(*) AS EmployeeCount,
    AVG(Salary) AS AverageSalary
FROM Employees
WHERE IsActive = 1
GROUP BY Department
HAVING AVG(Salary) > 70000;
```

Conceptually:

EXAMPLE / FLOW

```
Employees
   ↓
WHERE
Keep active employees
   ↓
GROUP BY
Create departments
   ↓
COUNT / AVG
Calculate summaries
   ↓
HAVING
Keep qualifying departments
```

Filtering Before Aggregation vs After Aggregation

Suppose the question is:

For active employees, show departments whose average salary is above $70,000.

Correct:

SQL

```
SELECT
    Department,
    AVG(Salary) AS AverageSalary
FROM Employees
WHERE IsActive = 1
GROUP BY Department
HAVING AVG(Salary) > 70000;
```

Here:

EXAMPLE / FLOW

```
WHERE
→ Who participates?

HAVING
→ Which resulting groups remain?
```

Can Aggregate Functions Be Used in WHERE?

Normally no.

Wrong:

SQL

```
SELECT
    Department,
    AVG(Salary)
FROM Employees
WHERE AVG(Salary) > 70000
GROUP BY Department;
```

`WHERE` executes logically before the group aggregate exists.

Correct:

SQL

```
HAVING AVG(Salary) > 70000
```

Can HAVING Be Used Without GROUP BY?

Yes.

If there is no `GROUP BY`, SQL Server can treat the qualifying result as one group.

Example:

```sql
SELECT COUNT(*) AS TotalEmployees
FROM Employees
HAVING COUNT(*) > 100;
```

But most commonly `HAVING` is used with `GROUP BY`.

WHERE vs HAVING Cheat Sheet

WHEREHAVING	
Filters rows	Filters groups
Before aggregation	After aggregation
Commonly uses normal columns	Often uses aggregates
Usually earlier filtering	Group-level filtering

Memory:

```
WHERE
→ Before GROUP BY

HAVING
→ After GROUP BY
```

Aggregate Functions and NULL

This deserves special attention.

Suppose:

EmployeeIdBonus	
1	1000
2	NULL
3	500
4	NULL

Results:

```sql
SELECT
    COUNT(*) AS TotalRows,
    COUNT(Bonus) AS BonusCount,
    SUM(Bonus) AS BonusTotal,
    AVG(Bonus) AS BonusAverage,
    MIN(Bonus) AS MinimumBonus,
    MAX(Bonus) AS MaximumBonus
FROM Employees;
```

Conceptually:

EXAMPLE / FLOW

```
COUNT(*)
→ 4

COUNT(Bonus)
→ 2

SUM(Bonus)
→ 1500

AVG(Bonus)
→ 750

MIN(Bonus)
→ 500

MAX(Bonus)
→ 1000
```

Most aggregate functions ignore NULL input values.

Be Careful Replacing NULL with Zero

These are not always equivalent:

SQL

```
AVG(Bonus)
```

and:

SQL

```
AVG(ISNULL(Bonus, 0))
```

Suppose:

EXAMPLE / FLOW

```
1000
NULL
500
NULL
```

`AVG(Bonus)`:

EXAMPLE / FLOW

```
1500 / 2
= 750
```

`AVG(ISNULL(Bonus,0))`:

EXAMPLE / FLOW

```
1500 / 4
= 375
```

Those mean completely different things.

Use zero only if:

EXAMPLE / FLOW

```
NULL really means zero
```

according to the business requirement.

Aggregating Calculated Expressions

Aggregate functions can contain expressions.

Example revenue:

SQL

```sql
SELECT
    SUM(
        Quantity * UnitPrice
    ) AS GrossRevenue
FROM OrderItems;
```

Revenue after discount:

SQL

```sql
SELECT
    SUM(
        Quantity * UnitPrice
        - ISNULL(DiscountAmount, 0)
    ) AS NetRevenue
FROM OrderItems;
```

AVG Calculated Expression

SQL

```sql
SELECT
    AVG(
        Quantity * UnitPrice
    ) AS AverageLineValue
FROM OrderItems;
```

This calculates each row's line value and then averages those calculated values.

Aggregate Functions with JOIN

Aggregation is commonly used after joining related tables.

Suppose:

EXAMPLE / FLOW

```
Customers
   ↓
Orders
```

We want:

Total order amount for each customer.

```sql
SELECT
    c.CustomerId,
    c.CustomerName,
    COUNT(o.OrderId) AS OrderCount,
    SUM(o.TotalAmount) AS TotalSpent
FROM Customers AS c
LEFT JOIN Orders AS o
    ON o.CustomerId = c.CustomerId
GROUP BY
    c.CustomerId,
    c.CustomerName;
```

Why LEFT JOIN May Be Important

With:

```sql
INNER JOIN
```

customers with no orders disappear.

With:

```sql
LEFT JOIN
```

customers with no orders can remain.

Example:

CustomerOrderCount	
John	5
Priya	0

Important COUNT Rule with LEFT JOIN

Consider:

```sql
COUNT(*)
```

after a `LEFT JOIN`.

A customer with no matching order still produces one outer-join row.

Therefore:

```sql
COUNT(*)
```

could return:

EXAMPLE / FLOW

```
1
```

even though the customer actually has zero orders.

Instead:

SQL

```
COUNT(o.OrderId)
```

counts only real matched orders because unmatched `o.OrderId` is NULL.

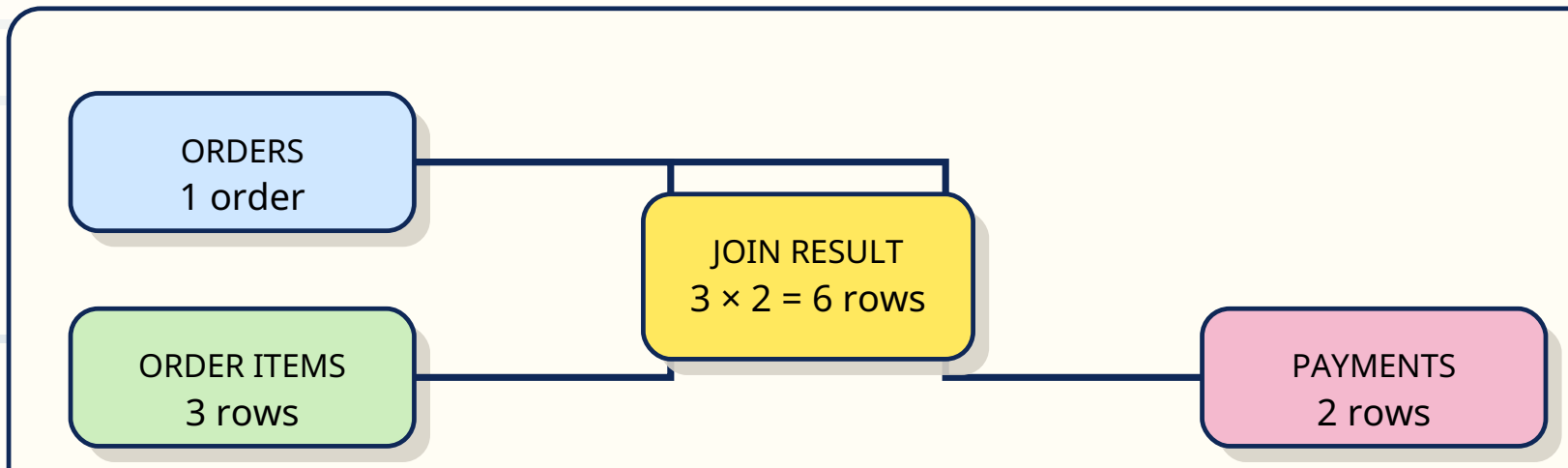
Correct pattern:

SQL

```
SELECT
    c.CustomerId,
    COUNT(o.OrderId) AS OrderCount
FROM Customers AS c
LEFT JOIN Orders AS o
    ON o.CustomerId = c.CustomerId
GROUP BY c.CustomerId;
```

This is an extremely important practical aggregation rule.

Σ Duplicate Rows Caused by JOINs



One-to-many joins can multiply rows and inflate SUM / COUNT / AVG

One of the most common aggregation mistakes is accidental row multiplication.

Suppose:

EXAMPLE / FLOW

```
Orders
→ OrderItems
→ Payments
```

An order has:

EXAMPLE / FLOW

```
3 OrderItems
2 Payments
```

Joining all three directly might produce:

EXAMPLE / FLOW

```
3 × 2 = 6 rows
```

The order total may then be counted several times.

This can incorrectly inflate:

EXAMPLE / FLOW

```
SUM
COUNT
AVG
```

How to Avoid Join Inflation

Understand the relationship cardinality before aggregating.

Sometimes you should aggregate child data first.

Example:

SQL

```sql
WITH PaymentTotals AS
(
    SELECT
        OrderId,
        SUM(Amount) AS TotalPaid
    FROM Payments
    GROUP BY OrderId
)
SELECT
    o.OrderId,
    pt.TotalPaid
FROM Orders AS o
LEFT JOIN PaymentTotals AS pt
    ON pt.OrderId = o.OrderId;
```

Now each order joins to only one payment summary row.

Common Aggregate Mistakes

Selecting a Column That Is Neither Grouped nor Aggregated

Wrong:

SQL

```sql
SELECT
    Department,
    EmployeeName,
    AVG(Salary)
FROM Employees
GROUP BY Department;
```

SQL Server doesn't know which employee name represents the department.

Using Aggregate Function in WHERE

Wrong:

SQL

```
WHERE AVG(Salary) > 70000
```

Use:

SQL

```
HAVING AVG(Salary) > 70000
```

Using HAVING When WHERE Would Be Better

This:

SQL

```
SELECT
    Department,
    COUNT(*)
FROM Employees
GROUP BY Department
HAVING Department = 'IT';
```

may work in some contexts, but:

SQL

```
WHERE Department = 'IT'
```

is conceptually better because SQL Server can filter rows before grouping.

Better:

SQL

```
SELECT
    Department,
    COUNT(*)
FROM Employees
WHERE Department = 'IT'
GROUP BY Department;
```

Rule:

If the condition applies to individual rows, prefer `WHERE`.

Confusing COUNT(*) with COUNT(column)

EXAMPLE / FLOW

```
COUNT(*)
→ rows

COUNT(column)
→ non-NULL values
```

Replacing NULL with Zero Without Business Reason

```sql
AVG(ISNULL(Value,0))
```

can completely change the meaning of the result.

Grouping Only by Month

This:

```sql
GROUP BY MONTH(OrderDate)
```

combines January from every year.

If multiple years exist, also group by year.

Overusing DISTINCT to Fix Bad Joins

If a join unexpectedly duplicates rows, developers sometimes add:

```sql
DISTINCT
```

to hide the problem.

That may hide incorrect logic rather than fix it.

Understand why rows are duplicated.

Aggregating the Wrong Granularity

Always ask:

> What should one result row represent?

Examples:

EXAMPLE / FLOW

```
One result per customer?

One result per month?

One result per product?

One result per department and city?
```

That answer should guide your `GROUP BY`.

Logical Idea of Aggregation

Consider:

SQL

```
SELECT
    Department,
    AVG(Salary)
FROM Employees
WHERE IsActive = 1
GROUP BY Department
HAVING AVG(Salary) >= 70000
ORDER BY Department;
```

For learning, think:

EXAMPLE / FLOW

```
FROM
↓
Get source rows

WHERE
↓
Filter rows

GROUP BY
↓
Create groups

Aggregate functions
↓
Calculate summaries

HAVING
↓
Filter groups

SELECT
↓
Return expressions

ORDER BY
↓
Sort final result
```

This explains why:

EXAMPLE / FLOW

```
WHERE
```

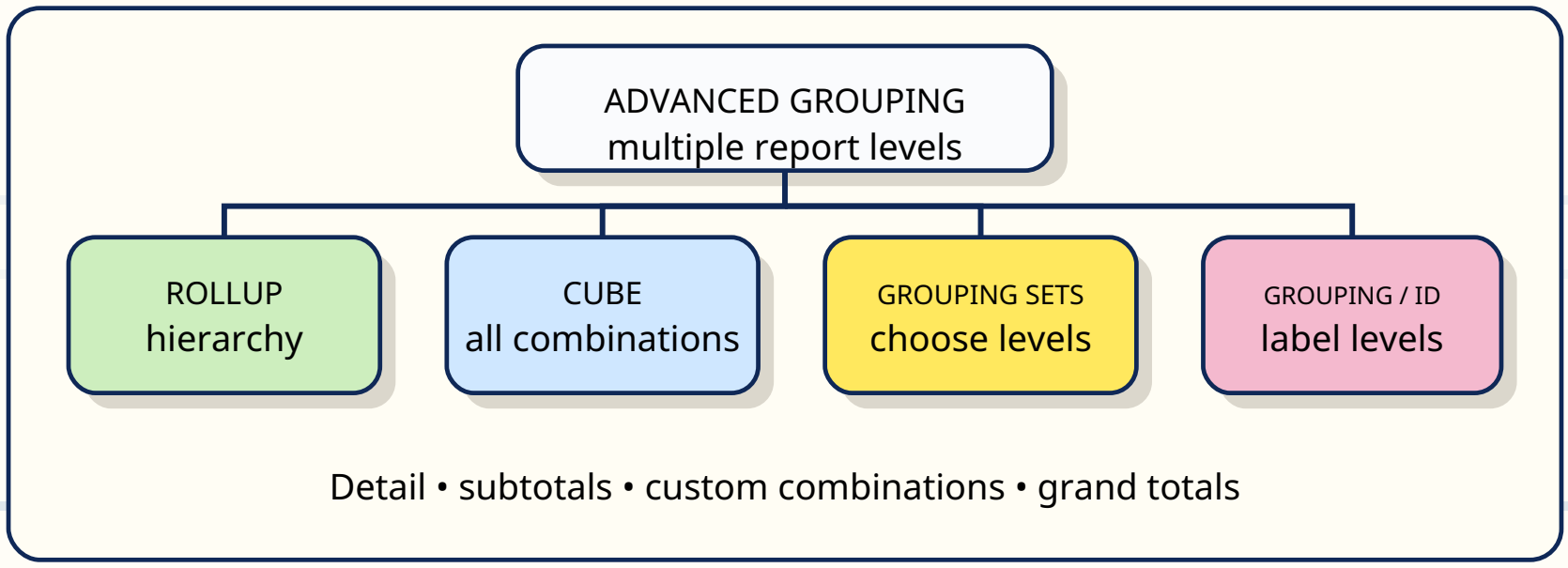
cannot normally filter:

EXAMPLE / FLOW

```
AVG(Salary)
```

because that aggregate doesn't exist yet at the WHERE stage.

Advanced Grouping Concepts



Normal `GROUP BY` gives one grouping level.

Example:

```sql
GROUP BY Region
```

returns totals per region.

Sometimes reports also need:

EXAMPLE / FLOW

```
Region totals
Product totals
Subtotals
Grand total
```

SQL Server provides:

EXAMPLE / FLOW

```
ROLLUP
CUBE
GROUPING SETS
GROUPING()
GROUPING_ID()
```

ROLLUP

Simple Definition

`ROLLUP` generates hierarchical subtotals and a grand total.

Suppose:

EXAMPLE / FLOW

```
Region
ProductCategory
SalesAmount
```

Query:

SQL

```
SELECT
    Region,
    ProductCategory,
    SUM(SalesAmount) AS TotalSales
FROM Sales
GROUP BY ROLLUP
(
    Region,
    ProductCategory
);
```

Conceptually it produces:

EXAMPLE / FLOW

```
Region + ProductCategory detail

Region subtotal

Grand total
```

ROLLUP Example Result

EXAMPLE / FLOW

```
East   Laptop    100000
East   Phone      50000
East   NULL      150000

West   Laptop     80000
West   Phone      40000
West   NULL      120000

NULL   NULL      270000
```

Here the generated `NULL` rows represent subtotal/grand-total levels.

ROLLUP Order Matters

For:

SQL

```
ROLLUP(Region, ProductCategory)
```

SQL Server conceptually generates:

EXAMPLE / FLOW

```
(Region, ProductCategory)

(Region)

()
```

It does not automatically generate a subtotal only by ProductCategory.

If you need every dimension combination, consider CUBE.

CUBE

Simple Definition

CUBE creates aggregates for **all combinations** of the grouping columns.

Example:

SQL

```
SELECT
    Region,
    ProductCategory,
    SUM(SalesAmount) AS TotalSales
FROM Sales
GROUP BY CUBE
(
    Region,
    ProductCategory
);
```

Conceptually it generates:

EXAMPLE / FLOW

```
Region + ProductCategory

Region only

ProductCategory only

Grand total
```

ROLLUP vs CUBE

For:

EXAMPLE / FLOW

```
Region
Product
```

ROLLUP:

EXAMPLE / FLOW

```
Region + Product

Region subtotal

Grand total
```

CUBE:

EXAMPLE / FLOW

```
Region + Product

Region subtotal

Product subtotal

Grand total
```

Memory:

EXAMPLE / FLOW

```
ROLLUP
→ Hierarchy

CUBE
→ All combinations
```

GROUPING SETS

Simple Definition

`GROUPING SETS` lets you explicitly specify exactly which grouping combinations you want.

Example:

SQL

```sql
SELECT
    Region,
    ProductCategory,
    SUM(SalesAmount) AS TotalSales
FROM Sales
GROUP BY GROUPING SETS
(
    (Region),
    (ProductCategory),
    ()
);
```

This requests:

EXAMPLE / FLOW

```
Region totals

Product category totals

Grand total
```

but not:

EXAMPLE / FLOW

```
Region + Product detail
```

unless you explicitly include it.

GROUPING SETS Example with Detail

SQL

```
GROUP BY GROUPING SETS
(
    (Region, ProductCategory),
    (Region),
    (ProductCategory),
    ()
)
```

This is conceptually equivalent to the combinations produced by a two-column CUBE.

Why GROUPING SETS Is Useful

Without it, you might write:

EXAMPLE / FLOW

```
Query 1
GROUP BY Region

UNION ALL

Query 2
GROUP BY Product

UNION ALL

Query 3
Grand total
```

GROUPING SETS lets you request those grouping levels in one grouping operation.

GROUPING()

Advanced grouping introduces an important problem.

Suppose:

EXAMPLE / FLOW

```
Region = NULL
```

can exist in your real data.

And ROLLUP also produces:

EXAMPLE / FLOW

```
Region = NULL
```

to represent a grand total.

How do you tell the difference?

Use:

SQL

```
GROUPING(Region)
```

GROUPING Function

`GROUPING(column)` returns:

EXAMPLE / FLOW

```
0
→ Column participates normally at this grouping level

1
→ NULL represents a super-aggregate level generated by
   ROLLUP/CUBE/GROUPING SETS
```

Example:

SQL

```sql
SELECT
    Region,
    GROUPING(Region) AS RegionGrouping,
    SUM(SalesAmount)
FROM Sales
GROUP BY ROLLUP(Region);
```

Labeling Grand Totals Properly

Instead of:

SQL

```sql
ISNULL(Region, 'TOTAL')
```

which cannot distinguish an actual NULL region from a generated total, use:

SQL

```sql
CASE
    WHEN GROUPING(Region) = 1
        THEN 'ALL REGIONS'

    WHEN Region IS NULL
        THEN 'UNKNOWN REGION'

    ELSE Region
END
```

This distinguishes:

EXAMPLE / FLOW

```
Real NULL value
```

from:

EXAMPLE / FLOW

```
ROLLUP/CUBE generated total
```

GROUPING_ID

`GROUPING_ID` provides an integer representing which columns were aggregated away at a grouping level.

Example:

SQL

```
SELECT
    Region,
    ProductCategory,

    GROUPING_ID(
        Region,
        ProductCategory
    ) AS GroupingLevel,

    SUM(SalesAmount) AS TotalSales

FROM Sales

GROUP BY ROLLUP
(
    Region,
    ProductCategory
);
```

For two columns, different result levels receive different numeric values.

This is useful when you need to:

EXAMPLE / FLOW

```
Identify grouping level

Label subtotal rows

Sort subtotal rows

Apply conditional report logic
```

GROUPING vs GROUPING_ID

EXAMPLE / FLOW

```
GROUPING(column)
→ Check one grouping column

GROUPING_ID(col1, col2, ...)
→ Identify combination/grouping level
```

Advanced Grouping Cheat Sheet

FeaturePurpose	
`GROUP BY`	Normal grouping
`ROLLUP`	Hierarchical subtotals + grand total
`CUBE`	All grouping combinations
`GROUPING SETS`	Choose exact group combinations
`GROUPING()`	Detect generated aggregate NULL

FeaturePurpose	
`GROUPING_ID()`	Identify grouping level

Complete Practical Example

Suppose:

EXAMPLE / FLOW

```
Orders
```

contains:

EXAMPLE / FLOW

```
OrderId
CustomerId
Region
OrderDate
Status
TotalAmount
```

We want:

For completed orders in 2026, show monthly sales by region, but only groups with at least 10 orders.

SQL

```
SELECT
    Region,

    DATEFROMPARTS
    (
        YEAR(OrderDate),
        MONTH(OrderDate),
        1
    ) AS OrderMonth,

    COUNT(*) AS OrderCount,

    COUNT(DISTINCT CustomerId)
        AS UniqueCustomers,

    SUM(TotalAmount)
        AS TotalRevenue,

    AVG(TotalAmount)
        AS AverageOrderValue,

    MIN(TotalAmount)
        AS MinimumOrderValue,

    MAX(TotalAmount)
        AS MaximumOrderValue

FROM Orders

WHERE
    Status = 'Completed'
    AND OrderDate >= '2026-01-01'
    AND OrderDate < '2027-01-01'

GROUP BY
    Region,

    DATEFROMPARTS
    (
        YEAR(OrderDate),
        MONTH(OrderDate),
        1
    )

HAVING COUNT(*) >= 10

ORDER BY
    OrderMonth,
    Region;
```

Understand the logic:

EXAMPLE / FLOW

```
WHERE
→ Keep completed 2026 orders

GROUP BY
→ Region + month

COUNT
→ Orders

COUNT DISTINCT
→ Different customers

SUM
→ Revenue

AVG
→ Average order

MIN
→ Smallest order

MAX
→ Largest order

HAVING
→ Remove groups with fewer than 10 orders

ORDER BY
→ Sort final report
```

Master Aggregate Cheat Sheet

RequirementFunction / Clause	
Count rows	`COUNT(*)`
Count populated values	`COUNT(column)`
Count unique values	`COUNT(DISTINCT column)`
Very large count	`COUNT_BIG`
Total	`SUM`
Average	`AVG`
Lowest	`MIN`
Highest	`MAX`
Combine strings	`STRING_AGG`
Sample standard deviation	`STDEV`
Population standard deviation	`STDEVP`
Sample variance	`VAR`
Population variance	`VARP`
Checksum summary	`CHECKSUM_AGG`
Create groups	`GROUP BY`
Filter rows first	`WHERE`
Filter groups afterward	`HAVING`

RequirementFunction / Clause	
Hierarchical totals	`ROLLUP`
Every grouping combination	`CUBE`
Custom grouping combinations	`GROUPING SETS`
Detect generated total	`GROUPING`
Identify grouping level	`GROUPING_ID`

Important Interview Questions

1. What is an aggregate function?

An aggregate function takes values from multiple rows and returns a summarized result, such as total, average, minimum, maximum, or count.

2. What is the difference between scalar and aggregate functions?

A scalar function normally returns one result for each input row. An aggregate function summarizes multiple rows into one result per group.

3. What is the difference between COUNT(*) and COUNT(column)?

`COUNT(*)` counts rows. `COUNT(column)` counts only rows where that column is not NULL.

4. What does COUNT(DISTINCT column) do?

It counts the number of different non-NULL values in the specified column.

5. How do aggregate functions handle NULL?

Most aggregates such as `SUM`, `AVG`, `MIN`, `MAX`, and `COUNT(column)` ignore NULL. `COUNT(*)` counts rows regardless of NULL values.

6. What is GROUP BY?

`GROUP BY` combines rows having the same grouping values so aggregate functions can calculate a summary for each group.

7. Can you SELECT a normal column that isn't in GROUP BY?

Normally no. A selected expression must either participate in the grouping or be aggregated because SQL Server needs one well-defined result per group.

8. What is the difference between WHERE and HAVING?

`WHERE` filters individual rows before grouping. `HAVING` filters groups after aggregate calculations have been performed.

9. Can aggregate functions be used directly in WHERE?

Normally no, because `WHERE` is logically processed before aggregation. Use `HAVING` to filter aggregate results.

10. Can HAVING be used without GROUP BY?

Yes. Without `GROUP BY`, the qualifying result can be treated as one aggregate group, although HAVING is most commonly used with GROUP BY.

11. Why can AVG(ISNULL(value,0)) give a different answer from AVG(value)?

`AVG(value)` ignores NULL values. Replacing NULL with zero adds extra zero values to the average and can change the business meaning significantly.

12. Why should we be careful with AVG on integer columns?

Integer input can produce integer-oriented results. Cast to an appropriate decimal type when fractional averages are required.

13. How do you calculate monthly totals?

Group by both the relevant year and month, or create a monthly date bucket. Grouping only by `MONTH(date)` mixes the same month from different years.

14. Can aggregate functions use calculated expressions?

Yes. For example, `SUM(Quantity * UnitPrice)` calculates each row's line amount and then totals those calculated values.

15. What is conditional aggregation?

Conditional aggregation combines `CASE` with functions such as `SUM` or `COUNT` to calculate different categories in one query.

Example: `SUM(CASE WHEN Status='Paid' THEN Amount ELSE 0 END)`.

16. Why can joins produce incorrect aggregate totals?

Joining multiple one-to-many tables can multiply rows. If aggregation happens afterward, amounts may be counted multiple times unless the join granularity is handled correctly.

17. What is STRING_AGG?

`STRING_AGG` combines string values from multiple rows into one delimited string, optionally with ordering.

18. What is the difference between STDEV and STDEVP?

`STDEV` calculates standard deviation for a sample. `STDEVP` calculates it for the complete population represented by the data.

19. What does ROLLUP do?

`ROLLUP` produces hierarchical subtotal rows and a grand total based on the order of grouping columns.

20. What is the difference between ROLLUP and CUBE?

`ROLLUP` follows a hierarchy of grouping levels. `CUBE` produces aggregates for every combination of its grouping columns.

21. What are GROUPING SETS?

`GROUPING SETS` explicitly specifies the exact group combinations required, allowing several different summary levels to be produced in one query.

22. Why is GROUPING() needed?

`ROLLUP`, `CUBE`, and `GROUPING SETS` may produce NULL to represent summary levels. `GROUPING()` lets you distinguish those generated NULL values from genuine NULL values in the source data.

23. What does GROUPING_ID do?

`GROUPING_ID` returns a numeric value representing which grouping columns have been aggregated away, making it useful for identifying subtotal and grand-total levels.

24. Why can COUNT(*) be wrong after a LEFT JOIN?

An unmatched parent row still exists as an outer-join result row. Use `COUNT(child.PrimaryKey)` when you want to count only actual matching child rows.

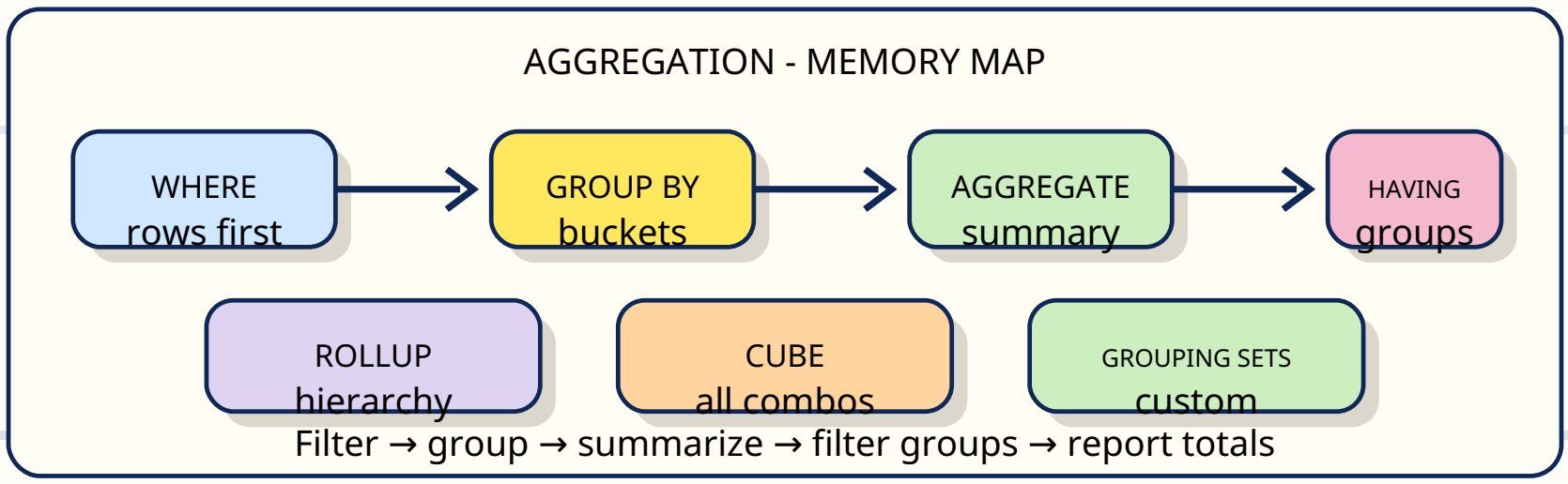
25. What is the most important question before writing GROUP BY?

Ask:

> **What should one output row represent?**

If one row should represent one department, group by department. If it should represent one department per month, group by department and month.

Final Quick Revision



EXAMPLE / FLOW

```
COUNT(*)
→ Count rows

COUNT(column)
→ Count non-NULL values

COUNT(DISTINCT column)
→ Count unique non-NULL values
```

EXAMPLE / FLOW

```
SUM
→ Total

AVG
→ Average

MIN
→ Lowest

MAX
→ Highest
```

EXAMPLE / FLOW

```
GROUP BY
→ Create groups

WHERE
→ Filter rows before grouping

HAVING
→ Filter groups after aggregation
```

EXAMPLE / FLOW

```
STRING_AGG
→ Multiple rows into one string
```

EXAMPLE / FLOW

```
STDEV / VAR
→ Sample statistics

STDEVP / VARP
→ Population statistics
```

EXAMPLE / FLOW

```
ROLLUP
→ Hierarchical subtotals

CUBE
→ All combinations

GROUPING SETS
→ Custom combinations

GROUPING
→ Is this a generated total?

GROUPING_ID
→ Which grouping level is this?
```

The easiest rule to remember is:

Aggregate functions summarize rows, `GROUP BY` decides which rows belong together, `WHERE` decides which rows participate, and `HAVING` decides which summarized groups remain.